

# 0\_Java Virtual Machine - 소개

## 1. JVM은 추상 컴퓨터다

공식 문서는 JVM을, 실제 컴퓨터처럼 **명령어 집합을 갖고 런타임에 여러 메모리 영역을 조작하는 추상적인 기계**로 정의한다.

- 물리적 CPU가 아니라, 소프트웨어로 구현된 가상의 CPU다.
- JVM은 자신만의 명령어 집합인 **바이트코드(bytecode)** 를 실행한다.

## 2. JVM은 자바 언어를 모른다

JVM은 자바 프로그래밍 언어에 대해 아무것도 모른다. 오직 바이너리 형식인 **클래스 파일 형식(class file format)**만 안다. 즉, JVM이 실제로 읽고 실행하는 것은 **.java**가 아니라 **.class** 파일이다.

**.class** 파일에는 바이트코드 명령어와 심볼 테이블 등이 담겨 있다.

여기서 중요한 결론이 나온다. **유효한 클래스 파일 형식으로 표현될 수 있는 기능이라면, 어떤 언어든 JVM 위에서 실행할 수 있다.** 대표적인 예가 Kotlin, Scala, Groovy다.

## 3. 전체 실행 파이프라인



자바 소스가 실행되기까지의 흐름은 대략 이렇다.

1. **작성**: 개발자가 **.java** 소스 파일을 작성한다.
2. **컴파일**: **javac**가 소스를 컴파일해 **.class** (바이트코드)로 바꾼다.
3. **로딩**: JVM의 클래스 로더가 **.class** 파일을 읽어 메모리에 올린다.
4. **검증**: 바이트코드 검증기가 유효하고 안전한 코드인지 확인한다.

5. **실행**: 실행 엔진이 바이트코드를 실행한다. 처음에는 인터프리터가 한 줄씩 해석하고, 자주 실행되는 부분은 JIT 컴파일러가 기계어로 컴파일해 성능을 끌어올린다.

핵심은, 개발자가 직접 다루는 것은 `.java` 와 `.class` 까지이고, 그 아래(로딩·검증·기계어 변환)는 전부 JVM이 책임진다는 점이다.

## 4. 플랫폼 독립성

JVM은 **하드웨어 및 운영체제의 독립성**을 책임지는 구성 요소다.

바이트코드는 특정 CPU나 OS에 종속되지 않은 형식이며, 각 플랫폼마다 그에 맞게 구현된 JVM이 존재한다. 그래서 **번역의 부담을 개발자가 아니라 플랫폼의 JVM이 진다**. 이것이 자바의 "Write Once, Run Anywhere"를 떠받치는 원리다.